

Praktikum Pemrograman Berbasis Objek

Pertemuan 2

Review

Live demo materi minggu lalu

Materi Pertemuan 2

Gambaran Materi Pembelajaran Hari Ini

Materi Pertemuan 2

1. Encapsulation

2. Data Hiding

3. Access Modifier

4. Constructor

5. Getter dan Setter

6. Data Integrity

Encapsulation

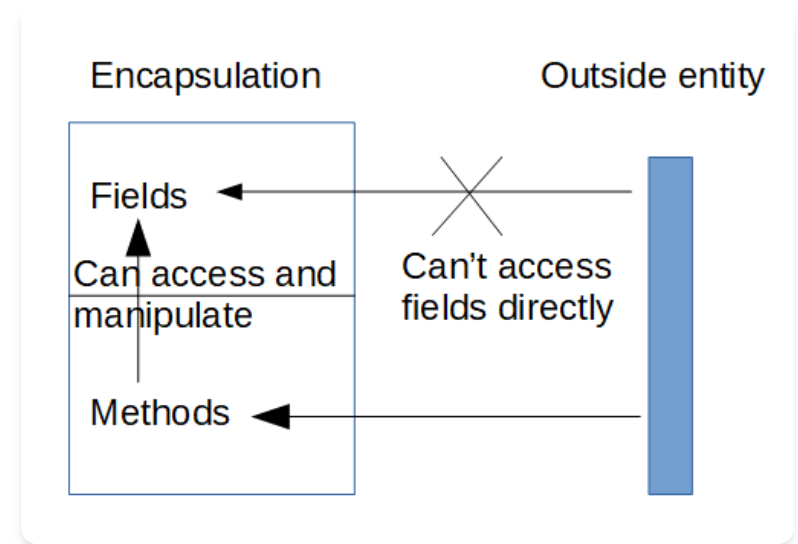
Konsep dan pentingnya penggunaan encapsulation di PBO

Encapsulation

Enkapsulasi merupakan salah satu konsep fundamental PBO, yaitu membungkus suatu kode dan data menjadi satu kesatuan. Salah satu keuntungan dari enkapsulasi adalah membuat suatu class hanya dapat diakses melalui class itu sendiri.

Syarat Enkapsulasi

- Mendeklarasi variabel sebagai `private`.
- Membuat method `public` setter dan getter untuk memodifikasi dan mengakses variabel tadi.



Kelebihan Encapsulation

- **Data Hiding**

Membatasi akses langsung ke variabel class, melindungi data sensitif.

- **Improved Maintainability**

Perubahan implementasi internal dapat dilakukan tanpa memengaruhi kode eksternal.

- **Enhanced Security**

Memungkinkan validasi dan kontrol atas data.

- **Code Reusability & Better Modularity**

Menjaga data dan method bersama dalam sebuah class.

Kekurangan Encapsulation

- **Increased Code Complexity**

Menulis getter dan setter untuk setiap variabel membuat kode lebih panjang dan lebih kompleks.

- **Performance Overhead**

Mengakses data melalui metode memiliki *minor performance cost*.

- **Less Flexibility**

Akses yang terlalu dibatasi dapat membatasi kemampuan class lain untuk menggunakan class tersebut secara efisien.

Data Hiding

Data hiding sebagai implementasi keamanan dalam enkapsulasi

Data Hiding

Data hiding merujuk pada praktik menjaga detail internal objek secara tersembunyi dari dunia luar dan hanya memberi akses terhadap fitur dan perilaku penting saja. Hal tersebut mencegah kode eksternal untuk memodifikasi state objek secara langsung yang dapat menyebabkan hal yang tak terduga atau korupsi data.

Data hiding diimplementasikan lewat **encapsulation**.

Implementasi di Java:

- **Private Fields:** Variabel yang menyimpan *state* objek dideklarasikan sebagai `private` agar tidak dapat diakses langsung dari luar class.
- **Public Getter and Setter Methods:** Metode ini memberikan akses terkontrol ke *private fields*. Getter mengambil nilai, sedangkan Setter memungkinkan modifikasinya.

Access Modifier

Berdasarkan apakah suatu class dapat diakses oleh class lain

Access Modifier

Access modifier membantu implementasi enkapsulasi dengan membatasi akses terhadap setiap bagian program yang berbeda.

	class yang satu package	class beda package
public	✓	✓
private	✗	✗
package (default)	✓	✗

Constructor

Method khusus dalam class

Constructor

Constructor adalah bagian khusus dari suatu class yang dipanggil ketika suatu objek sedang dibuat. Fungsinya adalah untuk menentukan atribut suatu objek baik secara default atau dari nilai yang ditentukan oleh user.

- Constructor bisa dibilang sebagai *special method* atau fungsi khusus dalam OOP.
- Nama dari constructor harus sama dengan nama classnya dan tidak memiliki return type.

Ada 2 tipe constructor, yaitu **default constructor** dan **regular constructor**.

```
class Mobil {  
    public Mobil() {  
        //isi kode disini  
    }  
}
```

Default Constructor

Default constructor tidak memiliki parameter apa pun dan digunakan untuk meng-assign nilai default ke suatu objek. Jika tidak ada constructor yang kita deklarasikan secara *explicit*, maka Java akan secara otomatis menyediakan *default constructor*.

```
class Mobil {  
    public void sound() {  
        System.out.println("Ngeeeng!");  
    }  
}  
  
public class Main {  
    public static void main(String [] args){  
        Mobil mobil1= new Mobil();  
        mobil1.sound(); //output: Ngeeeng!  
    }  
}
```

Regular Constructor

Regular constructor memiliki parameter untuk menginisialisasikan nilai-nilai dari atribut objek berdasarkan input yang telah kita tentukan.

```
class Mobil {  
    private String merek;  
    private int kapasitas;  
  
    // Regular Constructor  
    public Mobil(String merek, int kapasitas) {  
        this.merek = merek;  
        this.kapasitas = kapasitas;  
    }  
  
    public void infoMobil() {  
        System.out.println("Merek: " + this.merek);  
        System.out.println("Kapasitas: " + this.kapasitas + "cc");  
    }  
}
```


Getter dan Setter

Untuk manage akses terhadap atribut bersifat private

Getter dan Setter

Salah satu syarat implementasi enkapsulasi adalah menerapkan getter dan setter untuk setiap atribut yang ada dalam suatu class. Getter dan setter dibuat agar kita bisa mengakses dan mengupdate nilai dari atribut dengan modifier `private`.

- **Syntax:** Diawali dengan `get` atau `set` dan diikuti oleh nama atributnya dengan huruf pertama kapital.
- **Getter:** Mengembalikan nilai dari suatu atribut.
- **Setter:** Mengubah nilai dari suatu atribut.

```
class Mobil {  
    private String merek;  
    private int kapasitas;  
  
    public String getMerek() {  
        return merek;  
    }  
  
    public void setMerek(String merek) {  
        this.merek = merek;  
    }  
  
    public int getKapasitas() {  
        return kapasitas;  
    }  
  
    public void setKapasitas(int kapasitas) {  
        this.kapasitas = kapasitas;  
    }  
}
```

Implementasi Getter Setter di Class Main

Contoh yang salah

```
public class Main {  
    public static void main(String [] args){  
  
        Mobil mobil1 = new Mobil();  
        mobil1.merek = "Ferrari"; // error  
        System.out.println(mobil1.merek); // error  
    }  
}
```

Contoh yang benar

```
public class Main {  
    public static void main(String [] args){  
  
        Mobil mobil1 = new Mobil();  
        mobil1.setMerek("Ferrari");  
        System.out.println(mobil1.getMerek()); // output: F  
    }  
}
```

Data Integrity

Data Integrity

Data Integrity merujuk pada akurasi, konsistensi, dan reliabilitas data yang dipertahankan selama siklus hidupnya dalam aplikasi Java. Pencapaian integritas data melibatkan mekanisme seperti validasi data, transaksi, dan *error handling* untuk mencegah korupsi data atau perubahan yang tidak disengaja.

Definisi

Memastikan bahwa data tetap benar dan dilindungi dari akses atau perubahan yang tidak sah.

Fungsi

Penting untuk mempertahankan data berkualitas tinggi, membantu membuat keputusan yang tepat, dan memastikan pengguna dapat mempercayai informasi yang diberikan.

Tipe

entity integrity (memastikan pengenalan unik), *referential integrity* (memastikan hubungan antar tabel valid), dan *domain integrity* (memastikan nilai data berada dalam rentang yang ditentukan).

Latihan Minggu 2

Simulasi Manajemen Toko

Soal 1

Buat program CLI berbasis menu interaktif:

1. Buat class entitas (`Product` dan `Customer`).
 - Class `Product` menyimpan: `nama_produk`, `kategori`, `harga`, `stok_awal` dan method `hitung_subtotal()`.
 - Class `Customer` menyimpan: `customer_id`, `nama_customer`.
2. Objek Customer: Buat objek `Customer` di method Main. ID dan nama dibebaskan.
3. Mekanisme Tambah Produk: Buatlah mekanisme menambahkan produk, dimana semua atribut di kelas Produk harus didapatkan dari input.

CONTOH OUTPUT (INPUT MENU: 1)

≡≡≡ SISTEM MANAJEMEN TOKO ≡≡≡

1. Tambahkan Produk
2. Tampilkan semua produk
3. Transaksi

Pilih menu : 1

=====

Nama: baju fardan

Kategori: barang bekas

Harga: 17500

Stok Awal: 1

Soal 2

Tambahkan fitur agar pemilik toko bisa melihat semua produk dan customer bisa membeli produk.

- Untuk menampilkan semua produk, cukup menampilkan semua produk yang telah ditambahkan admin.
- Transaksi dimulai dari memasukkan `customer_id` , verifikasi (mencocokkan `customer_id` inputan dengan yang sudah dibuat di Main), memasukkan `nama_produk` dan kuantitas produk yang mau dibeli, dan menampilkan subtotalnya.

CONTOH OUTPUT

```
//Input: 2
```

```
≡≡≡ SISTEM MANAJEMEN TOKO ≡≡≡
```

1. Tambahkan Produk
2. Tampilkan semua produk
3. Transaksi

```
Pilih menu : 2
```

```
≡≡≡ LIST BARANG≡≡≡
```

```
Nama: baju fardan
```

```
Kategori: barang bekas
```

```
Harga: 17500
```

```
Stok Awal: 1
```